

Ports & Port Scanning

A complete, plain-language roadmap — expanded edition — from "what is a port?" to professional penetration-testing methodology, with every original analogy, table, and example preserved and extra explanation added throughout.

■ Stay Legal: Everything in this guide — scanning, enumeration, exploitation concepts — must only ever be done against systems you own or have written, explicit permission to test (like your own home lab, a Capture-The-Flag box, or a client who signed a contract). Scanning or attacking systems you don't own is illegal in most countries, full stop.

Roadmap Contents

- 1 1. Networking Basics — IP, TCP/UDP, DNS, Routers, Firewalls
- 2 2. Ports — 0–65535, Open/Closed/Filtered, Common Ports & Services
- 3 3. Linux Networking Tools — ip, ss, ping, curl, nc, dig
- 4 4. Build a Legal Lab — PC, VM, Docker, or Termux
- 5 5. Nmap Basics — Host Discovery, Port Scanning, Service/Version Detection
- 6 6. Manual Testing — Netcat, curl, Understanding Protocols
- 7 7. Packet Analysis — Wireshark, TCP Flags, Handshakes, Packets
- 8 8. Firewalls — Why Ports Become Filtered/Blocked
- 9 9. Advanced Nmap — TCP/UDP Scans, OS Detection, NSE, IPv6
- 10 10. Enumeration — Turning Open Ports Into Attack Surface Understanding
- 11 11. Vulnerability Assessment — CVEs, Misconfigurations, Weak Services
- 12 12. Automation & Professional Skills — Scripting, Reporting, Methodology

1. Networking Basics — IP, TCP/UDP, DNS, Routers, Firewalls

Before you can understand "ports," you need to understand the road the traffic drives on. This chapter covers the five basic building blocks of every network conversation on the internet.

1.1 What is an IP address?

Every device on a network needs an address so other devices know where to send information — just like a house needs a street address so the mailman knows where to deliver a letter. This address is called an IP address (Internet Protocol address).

■ **Donkey Analogy:** Think of the whole internet as a giant city. Every house (device — your phone, laptop, a website's server) has a house number (IP address). Without a house number, mail (data) has nowhere to go.

There are two versions in use today:

IPv4 — looks like 192.168.1.10. Four numbers (0–255) separated by dots. This is the older, more common format, but the world is running out of these addresses.

IPv6 — looks like 2001:0db8:85a3:0000:0000:8a2e:0370:7334. A much longer format created because IPv4 was running out of room.

Public vs Private IP addresses

Type	Example	Meaning
Private	192.168.x.x, 10.x.x.x, 172.16–31.x.x	Used inside your home/office network only. Like an internal apartment number that only makes sense inside the building.
Public	8.8.8.8, 142.250.premium	Visible to the whole internet. Like your building's actual street address.

Extra context: Because private IP ranges repeat inside millions of separate networks, your router performs a translation step (NAT — Network Address Translation) so that all the private devices behind it share one public IP when talking to the outside internet. This is also part of why home devices aren't directly reachable from the internet by default — NAT hides them behind the router's single public address.

1.2 TCP vs UDP — the two ways to "talk"

Once a message knows where it's going (the IP address), it still needs a way to actually be sent. There are two main delivery methods:

TCP (Transmission Control Protocol)

TCP is a reliable, connection-based protocol. Before any real data is sent, both sides do a little handshake to agree they're both ready — then data is sent in order, and if a piece goes missing, it gets resent.

■ **Donkey Analogy:** TCP is like a phone call. You dial, the other person says "hello?", you say "hi it's me" — then you start talking. If you don't hear them clearly, you say "sorry, what?" and they repeat it. Nothing gets lost because you're both confirming each part of the conversation.

UDP (User Datagram Protocol)

UDP is fast but unreliable. There's no handshake, no confirmation, no resending lost pieces. You just fire the data and hope it arrives.

■ **Donkey Analogy:** UDP is like shouting a message across a noisy party and walking away. Maybe they heard you, maybe they didn't — you're not waiting around to find out. It's fast because you skip all the "did you get that?" checking.

This is why video calls and online games use UDP (speed matters more than perfection — a dropped video frame is fine) while file transfers, web browsing, and email use TCP (accuracy matters — you can't have half a file).

Extra context: This distinction matters directly for scanning later on — TCP's handshake gives Nmap a clean, reliable signal for open/closed, while UDP's lack of a handshake is exactly why UDP scans (Chapter 9) are slower and much less certain.

1.3 DNS — the internet's phonebook

Computers navigate the internet using IP addresses (numbers), but humans are bad at remembering numbers. DNS (Domain Name System) translates human-friendly names like google.com into machine-friendly IP addresses like 142.250.80.14.

■ **Donkey Analogy:** DNS is a phonebook. You know your friend's name ("Google"), but the phone system actually needs their phone number ("142.250.80.14") to connect the call. DNS looks up the name and hands back the number.

1.4 Routers — the traffic directors

A router is a device that moves data between networks — for example, between your home network and the wider internet. It looks at where a piece of data is trying to go and decides the best path to send it.

■ **Donkey Analogy:** A router is a traffic cop standing at a busy intersection, pointing each car (piece of data) down the correct road toward its destination.

1.5 Firewalls — the bouncers

A firewall is a security guard that decides which traffic is allowed in or out of a network, usually based on rules like "allow web traffic, block everything else."

■ **Donkey Analogy:** A firewall is a nightclub bouncer with a guest list. Traffic that's on the list gets in; everything else gets turned away at the door.

Putting it together: You type google.com → DNS looks up its IP address → your router sends your request out onto the internet → it hops through many routers → arrives at Google's server → the firewall there checks if your request is allowed → TCP handles making sure the web page arrives complete and in order.

2. Ports — 0–65535, Open/Closed/Filtered, Common Ports & Services

2.1 What is a port, actually?

An IP address gets your data to the correct building (device). But a building has many rooms, each doing something different — one room does email, another does web pages, another does file transfer. A port is the "room number" that tells the data which specific service inside the device it needs to reach.

■ **Donkey Analogy:** If the IP address is the apartment building's street address, the port number is the apartment number. The mail (data) can find the right building easily, but without an apartment number, it doesn't know which door to knock on inside.

Ports are numbered from 0 to 65535 (that's 65,536 total — because computers count starting at 0). They're split into three ranges:

Range	Name	Meaning
0–1023	Well-Known Ports	Reserved for major, standard services (web, email, etc.) — assigned by an official authority (IANA).
1024–49151	Registered Ports	Used by specific applications that have registered them, but not as tightly controlled.
49152–65535	Dynamic/Private Ports	Used temporarily — e.g. your browser picks a random one of these to send a request from.

2.2 Open, Closed, and Filtered — the three states

When you check a port, it can be in one of three "states." Understanding these is the entire foundation of port scanning.

State	What it means	Donkey version
Open	Something is actively listening on that port and will respond/accept a connection.	You knock on the apartment door and someone opens it and says "yes?"
Closed	The device is reachable, but nothing is listening on that specific port — it actively replies "nobody's home here."	You knock on the door, and a neighbor pokes their head out and says "there's no one living in that apartment."
Filtered	A firewall or filter is blocking the request entirely — you get no response at all (or one that gives no useful info), so you can't tell if it's open or closed.	You knock on the door but there's a soundproof wall in front of it — you don't even know if anyone's inside because your knock never reaches the door.

Extra context: These three states map directly onto real network behavior: open = a service replies SYN-ACK (TCP) or sends data back (UDP); closed = the device sends an explicit rejection (TCP RST, or an ICMP "port unreachable" for UDP); filtered = a firewall drops the packet with no reply, or an intermediate device sends back an ICMP error that gives no real answer. You'll see this again in Chapters 7 and 8.

2.3 Common ports & services you MUST memorize

Port	Protocol	Service	What it's for
20/21	TCP	FTP	File Transfer Protocol — old-school file transfer.
22	TCP	SSH	Secure Shell — remote command-line login to a server.
23	TCP	Telnet	Old, unencrypted remote login (insecure, rarely used today).
25	TCP	SMTP	Sending email between mail servers.
53	TCP/UDP	DNS	Domain name lookups (the "phonebook" from Chapter 1).
67/68	UDP	DHCP	Automatically assigns IP addresses to devices on a network.
80	TCP	HTTP	Unencrypted web pages.
110	TCP	POP3	Older protocol for downloading email.
143	TCP	IMAP	Protocol for reading email while it stays on the server.
443	TCP	HTTPS	Encrypted web pages (HTTP + TLS/SSL encryption).
445	TCP	SMB	Windows file/printer sharing.
3306	TCP	MySQL	MySQL database connections.
3389	TCP	RDP	Remote Desktop Protocol — remote graphical login to Windows.
5432	TCP	PostgreSQL	PostgreSQL database connections.
8080	TCP	HTTP-alt	Common alternate port for web servers/proxies.

■ **Tip:** Don't try to memorize all 65,536 ports. Learn these ~15 first — they cover the vast majority of what you'll see in the real world. You'll pick up the rest naturally as you practice.

3. Linux Networking Tools — ip, ss, ping, curl, nc, dig

Before scanning other machines, you need to be comfortable poking around your own machine's network settings from the command line. These six tools are your daily-driver toolkit.

3.1 ip — see and configure network interfaces

Replaces the old ifconfig command. Shows your device's own IP addresses, network interfaces (Wi-Fi card, Ethernet card, etc.), and routing table.

```
ip addr show # Show all IP addresses on this machine
ip route show # Show the routing table (where traffic goes)
ip link show # Show network interfaces and their status (up/down)
```

■ **Donkey Analogy:** The ip command is you checking your own house's address plaque and the map of roads leading away from your house.

3.2 ss — see active connections and listening ports

ss (socket statistics) shows which ports are currently open and listening on your own machine, and what's connected to what.

```
ss -tulwn
# -t = TCP, -u = UDP, -l = listening only, -n = show numbers not names
```

■ **Donkey Analogy:** ss is you walking through your own house and checking which doors are currently unlocked and which rooms have people in them.

3.3 ping — is anybody home?

ping sends a tiny "are you alive?" packet (ICMP) to a target and waits for a reply. It confirms basic reachability and roughly how fast the connection is.

```
ping google.com
ping -c 4 192.168.1.1 # send exactly 4 pings then stop
```

■ **Donkey Analogy:** Ping is you knocking on someone's front gate and yelling "hello, anyone there?" — you're not trying to go inside, just checking if the house is occupied.

■ **Tip:** Some firewalls block ping on purpose, so "no reply" doesn't always mean "no one's home" — it might just mean the bouncer isn't answering that particular question.

3.4 curl — talk to web services directly

curl lets you manually send HTTP(S) requests from the terminal — great for testing web servers and APIs without opening a browser.

```
curl https://example.com # fetch a webpage
curl -I https://example.com # fetch only the HTTP headers
curl -v https://example.com # verbose mode – show the full conversation
```

■ **Donkey Analogy:** Curl is you walking up to the reception desk yourself and asking your question directly, instead of sending a fancy assistant (the browser) to ask for you.

3.5 nc (netcat) — the network "Swiss army knife"

nc can open raw connections to any port, listen on a port, or transfer data. It's the tool you use to manually "talk" to whatever's running behind an open port.

```
nc -v example.com 80 # try to connect to port 80 (verbose)
nc -lvp 4444 # listen on port 4444 (on your own machine)
```

■ **Donkey Analogy:** Netcat is a plain telephone with no fancy features — you can dial any extension (port) and just start talking, with nothing translating or dressing up the conversation for you.

3.6 dig — dig into DNS records

dig queries DNS servers directly and shows you the raw results — useful for troubleshooting or investigating a domain.

```
dig example.com # get the A record (IP address)
dig example.com MX # get the mail server records
dig example.com +short # short, clean output
```

■ **Donkey Analogy:** Dig is calling the phonebook company directly and asking, "give me the exact raw entry you have for this name" — instead of just looking the name up casually yourself.

■ **Tip:** Practice combo: Run **ip addr** to find your own IP → **ping** your router → **dig** a website's IP → **curl -I** that same website → **ss -tulwn** to see what's listening on your own machine. That one exercise touches every tool in this chapter.

4. Build a Legal Lab — Your PC, VM, Docker, or Termux

■ **Stay Legal:** This is the single most important chapter. Never point scanning tools at anything you don't own or don't have written permission to test. A home lab lets you learn everything safely and legally.

4.1 Why build a lab at all?

You need real, live open/closed/filtered ports to practice against. A lab gives you full control: you can open ports, close them, break things, and fix them — over and over — with zero legal risk and zero risk to anyone else's systems.

4.2 Option A — Virtual Machines (VMs)

A VM is a "computer inside your computer" — a fully isolated fake PC running inside a program on your real PC.

■ **Donkey Analogy:** A VM is like a dollhouse sitting on your kitchen table. It looks and behaves like a real house, but it lives entirely inside your real house and can't affect anything outside it.

VirtualBox or VMware Workstation Player — free tools to run VMs.

Kali Linux — a Linux distribution pre-loaded with security tools (Nmap, Wireshark, etc.) — this is your "attacker" machine.

Metasploitable2 / OWASP Broken Web Apps — intentionally vulnerable VMs made specifically for practicing on — these are your "target" machines.

Set both VMs on the same "Host-Only" or "Internal" virtual network so they can talk to each other, but not to the wider internet, and definitely not accessible from other real devices.

4.3 Option B — Docker containers

Docker is a lighter-weight way to spin up isolated services quickly — great for spinning up a target with one open service (like a database or web app) without an entire operating system.

```
docker run -d -p 8080:80 vulnerables/web-dvwa
# Runs a deliberately vulnerable web app, reachable at localhost:8080
```

■ **Donkey Analogy:** If a VM is a whole dollhouse, a Docker container is just a single dollhouse room — quicker to set up, using fewer resources, but with less flexibility.

4.4 Option C — Termux (practicing from Android)

Termux is a terminal emulator for Android that lets you install many Linux command-line tools (including Nmap) directly on your phone.

```
pkg install nmap
nmap 127.0.0.1
```

■ **Tip:** Termux is great for learning commands on the go, but a real VM lab is far better for building full target environments to practice against.

4.5 A simple, safe starter lab

1 Install VirtualBox on your PC.

- 2 Download and install Kali Linux as one VM (your "attacker box").
- 3 Download and install Metasploitable2 as a second VM (your "target box").
- 4 Set both VM network adapters to "Host-Only Adapter" — this keeps them isolated from the real internet.
- 5 Boot both machines, find each VM's IP with **ip addr**, and confirm they can ping each other.

Now you have a completely legal, fully isolated playground with real open ports and real vulnerable services to practice everything in this guide against.

5. Nmap Basics — Host Discovery, Port Scanning, Service/Version Detection

5.1 What is Nmap?

Nmap ("Network Mapper") is the industry-standard tool for discovering devices on a network and finding out what ports and services they're running. It's the single most important tool in this entire roadmap.

■ **Donkey Analogy:** Nmap is a very thorough building inspector who walks around a building, knocks on every door, notes which ones open, and even asks "what's your job?" to whoever answers.

5.2 Host discovery — "who's actually alive on this network?"

```
nmap -sn 192.168.1.0/24
# -sn = "ping scan" — just find live hosts, don't scan ports
```

This sends a lightweight probe to every address in that range (192.168.1.0/24 means all addresses from 192.168.1.0 to 192.168.1.255) and lists which ones respond.

5.3 Basic port scanning

```
nmap 192.168.1.50
# Default scan: checks the 1000 most common TCP ports
```

Output looks roughly like this:

```
PORT STATE SERVICE
22/tcp open  ssh
80/tcp open  http
3306/tcp closed mysql
```

Useful flags to control scope:

Flag	Meaning
-p 80,443	Scan only specific ports.
-p-	Scan ALL 65535 ports (slower, but thorough).
-F	Fast scan — only the 100 most common ports.

5.4 Service and version detection

```
nmap -sV 192.168.1.50
```

The -sV flag tells Nmap to not just check if a port is open, but to actually poke it and try to identify what software and version is running behind it (e.g. "Apache httpd 2.4.29" instead of just "port 80 open").

■ **Donkey Analogy:** A basic scan tells you "there's a person behind Door 22." Version detection asks that person "hey, what's your name and how old are you?" — and they answer "I'm SSH, version OpenSSH 8.2."

5.5 Combining flags — a solid everyday command

```
nmap -sV -sC -p- -T4 192.168.1.50
```

- sV** — detect versions
- sC** — run Nmap's default safe scripts (basic enumeration)
- p** — scan all 65535 ports
- T4** — faster timing template (T0 = slowest/stealthiest, T5 = fastest/loudest)

■ **Tip:** This is close to the "go-to" command most professionals run first against a new target in an authorized engagement.

6. Manual Testing — Netcat, curl, Understanding Protocols

Nmap tells you a port is open and guesses what's running there. But to really understand a service, you should manually connect to it yourself and see the raw conversation. This is called "banner grabbing" and manual protocol interaction.

6.1 Manual banner grabbing with Netcat

```
nc -v 192.168.1.50 22
```

Many services immediately introduce themselves the moment you connect — this is the "banner." For example, connecting to an SSH port might immediately show:

```
SSH-2.0-OpenSSH_8.2p1 Ubuntu-4ubuntu0.5
```

■ **Donkey Analogy:** This is like picking up the phone and, before you even say a word, the person on the other end announces "Hi, this is Dave from Accounting" — you now know exactly who you're talking to without asking.

6.2 Manually talking HTTP with Netcat

HTTP is a plain-text protocol, so you can type it by hand:

```
nc -v example.com 80
GET / HTTP/1.1
Host: example.com
(press Enter twice)
```

You'll get the raw HTTP response headers and page content back — exactly what a browser receives, just without any of the pretty rendering.

6.3 The same test with curl (easier)

```
curl -v http://example.com/
```

curl -v shows you the full request it sent and the full response it got back, which is a faster way to see the same conversation Netcat shows you manually.

6.4 Why bother doing this manually at all?

Automated tools can misidentify services, especially unusual or custom ones. Manually connecting and reading the raw banner/response teaches you to recognize protocols by sight, and it's often the fastest way to confirm exactly what version and configuration a service is really running.

7. Packet Analysis — Wireshark, TCP Flags, Handshakes, Packets

7.1 What is a packet?

Data traveling over a network isn't sent as one giant blob — it's chopped into small chunks called packets, each with its own little "envelope" of information (source IP, destination IP, port numbers, etc.) attached to the front.

■ **Donkey Analogy:** Sending a long letter over the internet is like tearing it into a hundred postcards, numbering each one, writing the same destination address on every single postcard, and mailing them all separately — the receiving end reassembles them in the right order.

7.2 Wireshark — the microscope for network traffic

Wireshark is a tool that captures every packet flowing across a network interface and lets you inspect each one in detail — every header, every flag, every byte.

■ **Donkey Analogy:** If Nmap is the building inspector, Wireshark is a security camera recording every single person who walks in or out of every door, with a full transcript of what they said.

7.3 The TCP three-way handshake

Remember from Chapter 1 that TCP is like a phone call — both sides confirm they're ready before real data flows. This confirmation is called the three-way handshake, and it uses TCP "flags" (small markers in the packet header):

- 1 **SYN** — Computer A says: "I'd like to start a connection" (SYN = synchronize).
- 2 **SYN-ACK** — Computer B replies: "Okay, got it, I'm ready too" (SYN-ACK = synchronize + acknowledge).
- 3 **ACK** — Computer A replies: "Great, let's go" (ACK = acknowledge).

■ **Donkey Analogy:** It's exactly like answering a phone call:

You: "Hello?" (SYN)

Them: "Hi, can you hear me?" (SYN-ACK)

You: "Yes, go ahead" (ACK)

Only after this three-step exchange does the actual conversation (data) begin.

7.4 Common TCP flags

Flag	Meaning
SYN	Start a new connection.
ACK	Acknowledge that data was received.
FIN	Politely finish/close the connection.
RST	Abruptly reset/refuse the connection (this is what a closed port sends back).
PSH	Push data through immediately, don't buffer it.

Flag	Meaning
URG	This data is urgent, prioritize it.

7.5 Why this matters for port scanning

Nmap's most common scan type (the "SYN scan," -sS) works by sending only the first SYN packet and watching the reply — without completing the handshake. A SYN-ACK reply means the port is open; an RST reply means closed; no reply at all (or an ICMP error) usually means filtered. Wireshark lets you literally watch this exchange happen, packet by packet, which is the best way to truly understand what a "scan" is doing under the hood.

8. Firewalls — Why Ports Become Filtered or Blocked

8.1 Recap: what makes a port "filtered"?

A port is filtered when a firewall (or similar filtering device) sits between you and the target and silently drops or blocks your probe, so you get no useful answer back.

8.2 How firewalls decide what to block

Firewalls follow rulesets — lists of conditions like "allow traffic to port 443 from anywhere," or "block all traffic to port 22 except from this one trusted IP address."

■ **Donkey Analogy:** The bouncer (firewall) from Chapter 1 has a clipboard of rules: "Let anyone through the Champagne door (443). Only let VIPs with a special wristband through the Staff door (22). Turn everyone else away silently — don't even tell them why."

8.3 Types of firewalls you'll encounter

Type	Where it lives	Example
Host-based firewall	On the individual computer itself	Windows Defender Firewall, iptables / ufw on Linux
Network firewall	Between whole networks, often on a router/dedicated appliance	Cisco ASA, pfSense, corporate firewall appliances
Web Application Firewall (WAF)	In front of a specific web application	Cloudflare, ModSecurity

8.4 Why a scan result can be "wrong" or misleading

- A firewall may silently drop packets (no reply at all) — this looks "filtered" in Nmap.
- A firewall may actively reject packets (send back an ICMP "unreachable" message) — this can also show as "filtered," but with a slightly different signature.
- Some firewalls do rate limiting — they'll answer your first few probes normally, then start dropping or slowing responses once they detect a scan pattern, making your results inconsistent.

■ **Tip:** When you see a lot of "filtered" results, that's not a dead end — it's actually useful intelligence: it tells you a firewall is actively defending that host, and which specific doors it's guarding most closely.

9. Advanced Nmap — TCP/UDP Scans, OS Detection, NSE, IPv6

9.1 The different TCP scan types

Flag	Name	How it works
-sS	SYN scan ("half-open")	Sends SYN, reads reply, never completes handshake. Fast and relatively stealthy (default when run as admin/root).
-sT	Connect scan	Completes the full handshake using the OS's normal networking calls. Slower, more visible in logs, but doesn't need special privileges.
-sA	ACK scan	Sends only an ACK packet — doesn't tell you open vs closed, but helps map firewall rules.
-sF / -sX / -sN	FIN/Xmas/Null scans	Send unusual flag combinations to try to sneak past simple firewalls (rely on quirks in how systems reply to malformed packets).

9.2 UDP scanning

```
nmap -sU -p 53,67,161 192.168.1.50
```

UDP scanning is trickier and slower than TCP scanning because UDP has no handshake — an open UDP port often sends back nothing, and Nmap can only be sure a port is "closed" if it gets back an explicit ICMP "port unreachable" error. This is why UDP scans take much longer and are less certain than TCP scans.

■ **Donkey Analogy:** TCP scanning is knocking on a door and clearly hearing "come in" or "no one's home." UDP scanning is like slipping a note under a door and getting no reply at all most of the time — silence could mean "they got it and are busy," or "no one lives there." You genuinely can't always tell.

9.3 OS detection

```
nmap -O 192.168.1.50
```

Nmap examines subtle quirks in how a device's TCP/IP stack responds (things like default packet TTL values, window sizes, and how it handles unusual packets) and compares them against a database of known operating system "fingerprints" to guess the OS.

9.4 NSE — the Nmap Scripting Engine

NSE lets Nmap run small automated scripts against open ports to do deeper checks — like grabbing more detailed banners, testing for common misconfigurations, or even checking for specific known vulnerabilities.

```
nmap --script=default -sV 192.168.1.50
nmap --script=vuln 192.168.1.50 # check for known vulnerabilities
nmap --script=http-title 192.168.1.50 # grab the title of a web page
```

■ **Donkey Analogy:** If a normal scan is knocking on doors, NSE scripts are little specialist assistants you send in behind you, each trained to ask one very specific extra question once a door opens.

9.5 Scanning IPv6 targets

```
nmap -6 2001:db8::1
```

The -6 flag tells Nmap to use IPv6 addressing. As more networks adopt IPv6, this becomes increasingly relevant — some devices only expose certain services over IPv6, so scanning only IPv4 can miss things.

10. Enumeration — Turning Open Ports Into Attack Surface Understanding

10.1 What is "enumeration"?

Scanning tells you which doors are open. Enumeration is the deeper process of walking through each open door and figuring out exactly what's inside — usernames, shared folders, software versions, configuration details — anything that expands your understanding of the target.

■ **Donkey Analogy:** Scanning finds the unlocked doors in a building. Enumeration is actually walking into each room, reading the labels on the filing cabinets, and noting down everyone's name tag.

10.2 What "attack surface" means

The attack surface is the total collection of all the ways a system could potentially be interacted with (and therefore attacked) — every open port, every running service, every login page, every user account.

Bigger attack surface = more possible weak points. Part of good defensive security is reducing attack surface: closing unused ports, disabling unnecessary services, removing old accounts.

10.3 Common enumeration targets by service

Service	What to enumerate	Example tool
FTP (21)	Anonymous login allowed? Files available?	ftp, Nmap NSE scripts
SSH (22)	Version, supported authentication methods, valid usernames	ssh, nmap -sV
SMB (445)	Shared folders, usernames, OS version	enum4linux, smbclient
HTTP/S (80/443)	Directories, technologies used, login pages, hidden pages	gobuster, whatweb, browser dev tools
DNS (53)	Subdomains, zone transfers	dig, dnsrecon

10.4 Example enumeration workflow

- 1 Run Nmap with -sV -sC to find open ports and grab basic info.
- 2 For each interesting port, manually connect (curl/netcat) to confirm the banner.
- 3 Run a service-specific enumeration tool (e.g. gobuster for a web server directory listing).
- 4 Note down every username, software version, and file path discovered.
- 5 Cross-reference software versions against known vulnerabilities (Chapter 11).

11. Vulnerability Assessment — CVEs, Misconfigurations, Weak Services

11.1 What is a CVE?

CVE stands for Common Vulnerabilities and Exposures — a public, standardized catalogue of known security flaws in software, each with a unique ID like CVE-2021-44228 (the famous "Log4Shell" bug).

■ **Donkey Analogy:** A CVE is like a recall notice for a car model — "2019 Toyota Camry, brake issue, recall number XYZ." It tells you exactly which product, which version, and what's wrong with it.

11.2 Matching services to CVEs

Once you know a service's exact name and version (from Nmap's -sV), you can search vulnerability databases to see if known flaws exist for that exact version.

NVD (National Vulnerability Database) — nvd.nist.gov

Exploit-DB — exploit-db.com

searchsploit — a command-line tool (comes with Kali) that searches Exploit-DB offline

```
searchsploit apache 2.4.29
```

11.3 Misconfigurations — often worse than software bugs

Many real-world breaches happen not because of a fancy unpatched vulnerability, but because of simple mistakes:

- Default or blank admin passwords left unchanged.
- Anonymous FTP or SMB access left enabled.
- Unnecessary services running that were never turned off.
- Overly permissive firewall rules ("allow everything from everywhere").
- Directory listing enabled on a web server, exposing files that shouldn't be public.

■ **Donkey Analogy:** A CVE is a broken lock the manufacturer forgot to fix. A misconfiguration is a perfectly good lock that someone simply forgot to lock.

11.4 Weak/outdated services

Some services are inherently risky no matter how they're configured, because they were designed decades ago without modern security in mind:

- **Telnet** — sends everything, including passwords, in plain unencrypted text.
- **FTP** — also unencrypted by default.
- **SMBv1** — old version of Windows file sharing, riddled with known critical vulnerabilities (e.g. EternalBlue / WannaCry).

11.5 Severity scoring — CVSS

Vulnerabilities are ranked using the CVSS (Common Vulnerability Scoring System), a score from 0.0 to 10.0 indicating how severe a flaw is, based on things like how easy it is to exploit and how much damage it could do.

Score	Severity
0.1–3.9	Low
4.0–6.9	Medium
7.0–8.9	High
9.0–10.0	Critical

■ Stay Legal: Finding a vulnerability is not permission to exploit it. In real, authorized engagements, exploitation is done carefully, within the exact scope agreed in writing, and often only enough to prove the flaw is real (not to cause damage).

12. Automation & Professional Skills — Scripting, Reporting, Methodology

12.1 Why learn scripting?

Manually running the same Nmap command, copy-pasting results, and checking each one by hand doesn't scale once you're testing dozens or hundreds of hosts. Scripting (Python or JavaScript) lets you automate repetitive parts of the process.

12.2 Simple Python automation example

```
import subprocess
targets = ["192.168.1.10", "192.168.1.20", "192.168.1.30"]
for ip in targets:
    result = subprocess.run(
        ["nmap", "-sV", "-oN", f"{ip}.txt", ip],
        capture_output=True, text=True
    )
    print(f"Scanned {ip}, results saved to {ip}.txt")
```

This loops through a list of target IPs, runs Nmap against each one, and saves each result to its own file — instead of you typing the same command over and over.

12.3 Parsing Nmap output programmatically

Nmap can output results as structured XML (-oX results.xml), which Python libraries like python-nmap or xml.etree.ElementTree can read and turn into clean data — perfect for feeding into your own dashboards or reports.

12.4 Writing a professional penetration-test report

A real engagement isn't finished until it's documented clearly enough for a non-technical manager and a technical engineer to both understand it. A good report typically includes:

- 1 **Executive Summary** — plain-language overview for management: what was tested, overall risk level, top priorities.
- 2 **Scope & Methodology** — exactly what was in-bounds, what tools/techniques were used, and the timeline.
- 3 **Findings** — each vulnerability listed individually with: description, affected host/port, CVSS/severity score, evidence (screenshots/output), and business impact.
- 4 **Remediation Recommendations** — clear, actionable steps to fix each finding.
- 5 **Appendix** — raw scan output, full tool logs, technical detail for engineers to reproduce findings.

12.5 A standard penetration-testing methodology

Professional testing generally follows a repeatable structure, often based on frameworks like PTES (Penetration Testing Execution Standard) or OSSTMM:

Phase	What happens
1. Pre-engagement	Agree scope, rules of engagement, and get signed written authorization.
2. Reconnaissance	Gather public info about the target (domains, IP ranges, employees).
3. Scanning & Enumeration	Everything from Chapters 5, 9, and 10 of this guide.
4. Vulnerability Assessment	Match findings against CVEs/misconfigurations (Chapter 11).
5. Exploitation	Carefully, within scope, prove that a vulnerability is real and exploitable.
6. Post-Exploitation	Assess what an attacker could reach next (only if explicitly authorized).
7. Reporting	Deliver the professional report described in 12.4.
8. Remediation & Re-test	Client fixes issues: tester verifies the fixes actually worked.

■ Stay Legal: Every single phase above assumes a signed contract and clearly defined scope exists before any scanning begins. Skipping this step turns a professional security exercise into a crime.

You made it through the roadmap. The real learning now happens by doing: rebuild the lab from Chapter 4, run every command in this guide against it, break things on purpose, and read the raw output slowly until it makes sense. Come back to this guide anytime you need a refresher.

This guide is for educational purposes to support ethical, authorized security learning and testing only.